

Accelerating Chip Design with the Open PDKs using GDSFACTORY

Joaquin Matres



DAC conference



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



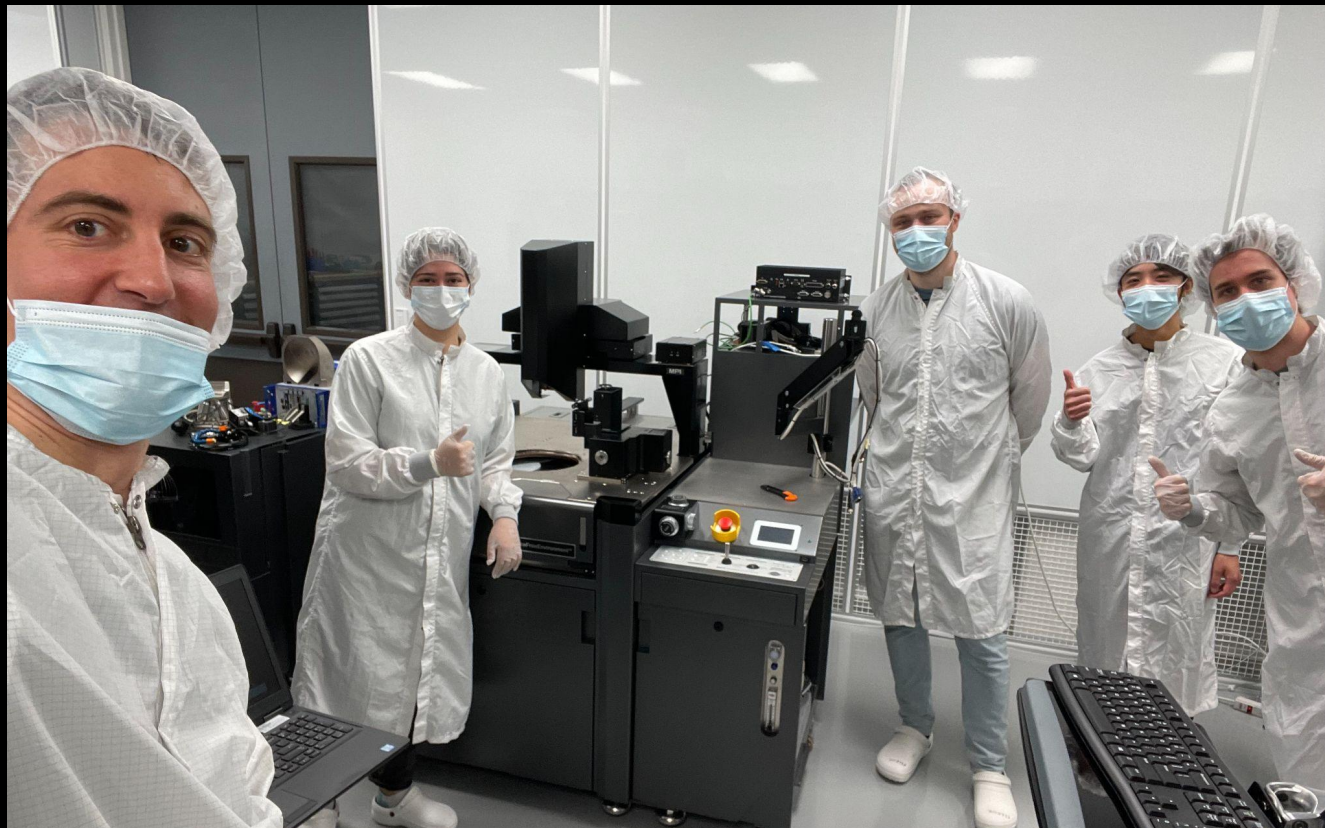
UC DAVIS







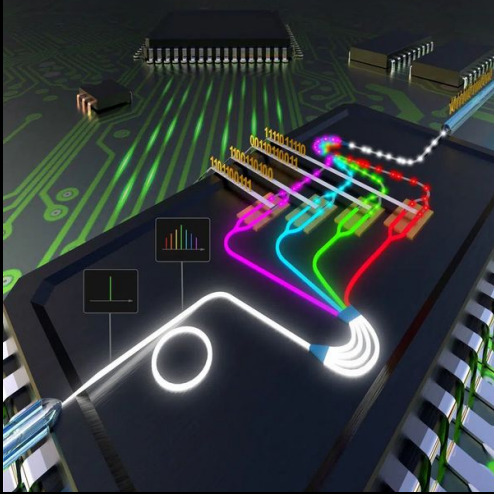




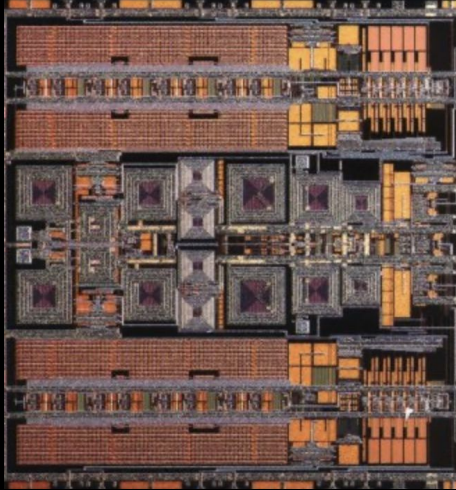


GDSFACTORY

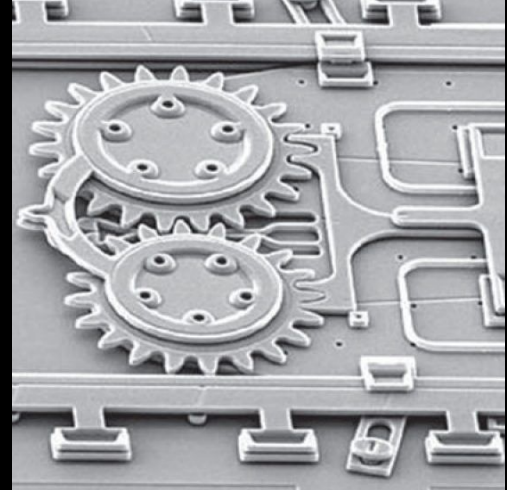
Make analog chips more accessible



Photonics



RF / Mixed Signal



MEMs

GDSFACTORY

+5M

Downloads

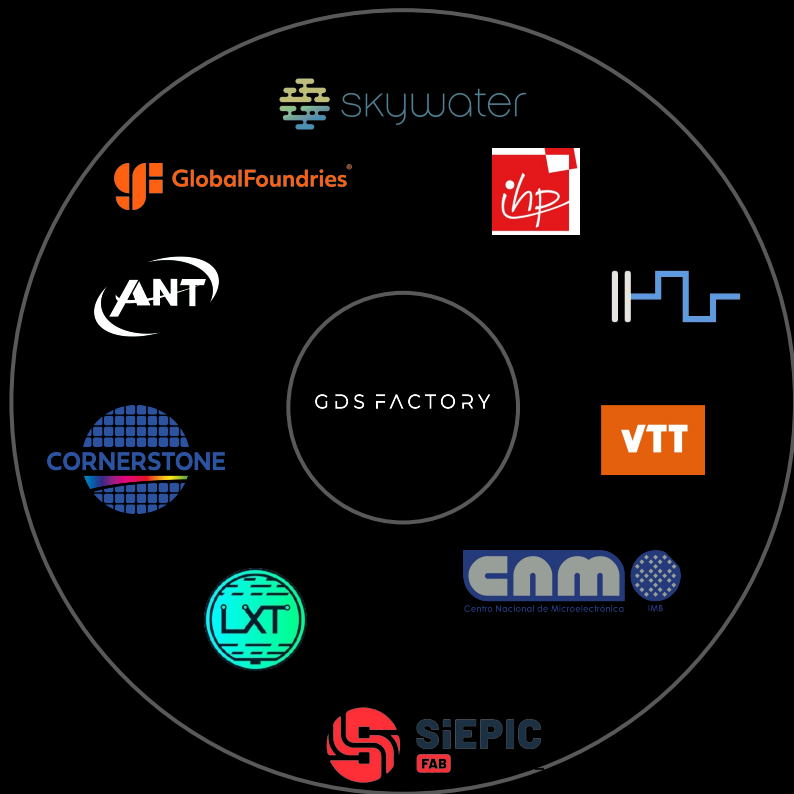
+122

Contributors

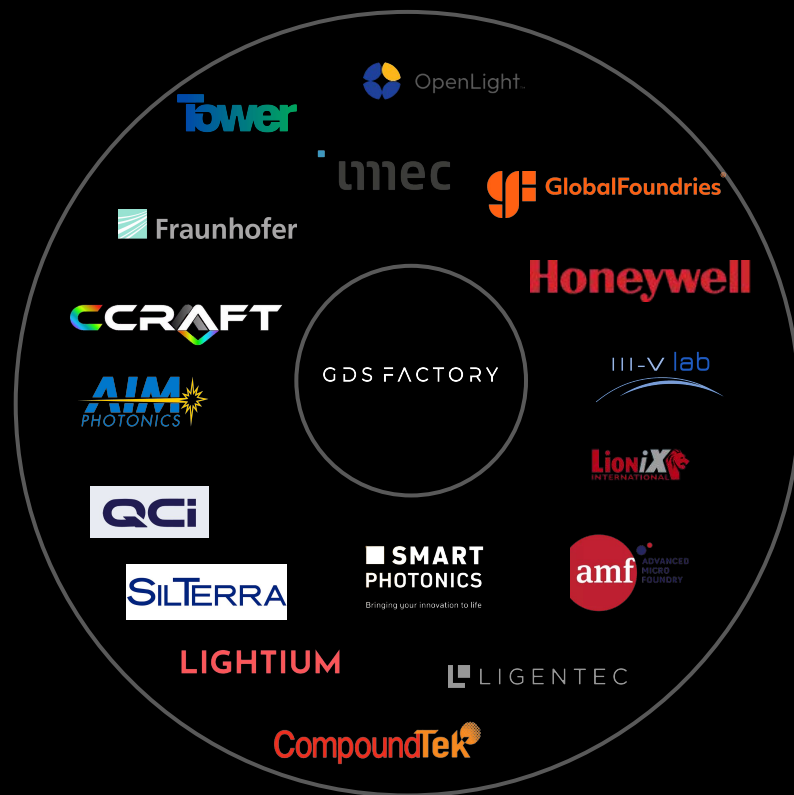
+50
PDKs

Foundry
support

10+ Open Source



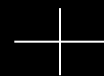
40+ Private



gdsfactory.com/plus/pdks

Tech

GDSFACTORY



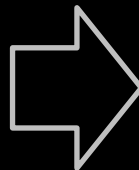
1. Python Code

```
import gdsfactory as gf

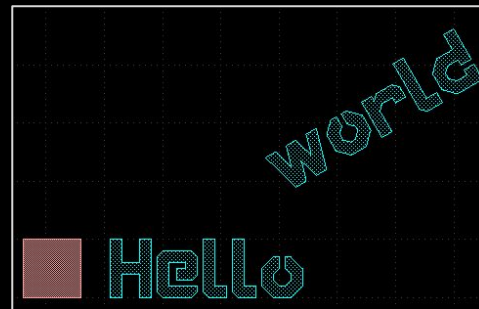
c = gf.Component()
ref1 = c.add_ref(gf.components.rectangle(size=(10, 10), layer=(1, 0)))
ref2 = c.add_ref(gf.components.text("Hello", size=10, layer=(2, 0)))
ref3 = c.add_ref(gf.components.text("world", size=10, layer=(2, 0)))

ref1.xmax = ref2.xmin - 5

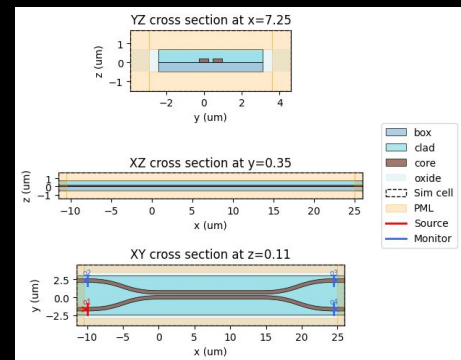
ref3.xmin = ref2.xmax + 2
ref3.rotate(30)
c.show()
```



2. Write CAD (GDS, OASIS, STL, GERBER)



3. Run simulations



A Different Approach

Layout is Code

Instead of drawing polygons manually:

```
from ihp import PDK
PDK.activate()

from ihp.cells import nmos
nmos(width=1.0, length=0.13, nf=8)
```

Instead of **Hundreds** of mouse clicks.



Manual & Error-prone



Code-driven & Scalable



Reproducible



Version controlled



Parameterized



Automated



AI-ready

Simulation Engines

From TCAD to FDTD – a complete toolbox for modern electronic design

TCAD

Semiconductor devices



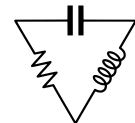
FEM

Finite-element multiphysics



SPICE

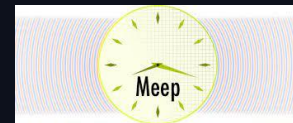
SPICE-class simulators



circuitlux

FDTD

Time-domain electromagnetics



FDTD X



BEAMZ

★ Unified interfaces • ★ Open-source ready • ★ Photonic, electronic, and quantum technologies

Feature

Legacy (SPICE)



circulax



**Model
Definition**



Hardcoded C++ /
Verilog-A



Simple python
functions



Derivatives



Hardcoded (C) or
Compiler-Generated
(Verilog-A)

AD

Automatic
Differentiation (AD)



Solver Logic



Fixed-step or
heuristic-based



Adaptive ODE
stepping via Diffrax



Matrix Solver



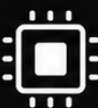
Monolithic CPU
Sparse (KLU)



Pluggable (KLUJAX,
Dense, or Custom)



**Hardware
Target**

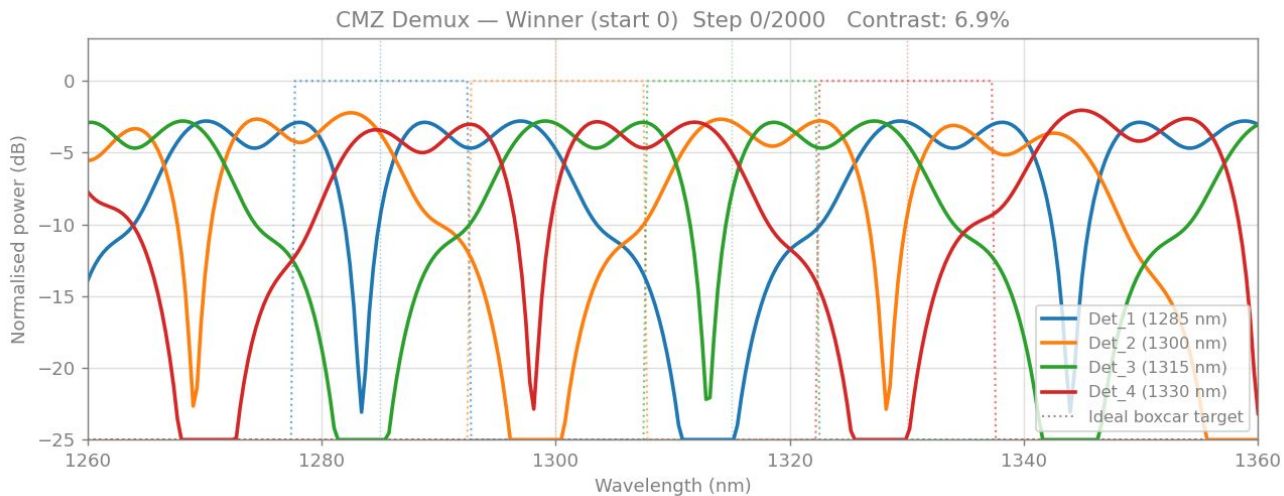
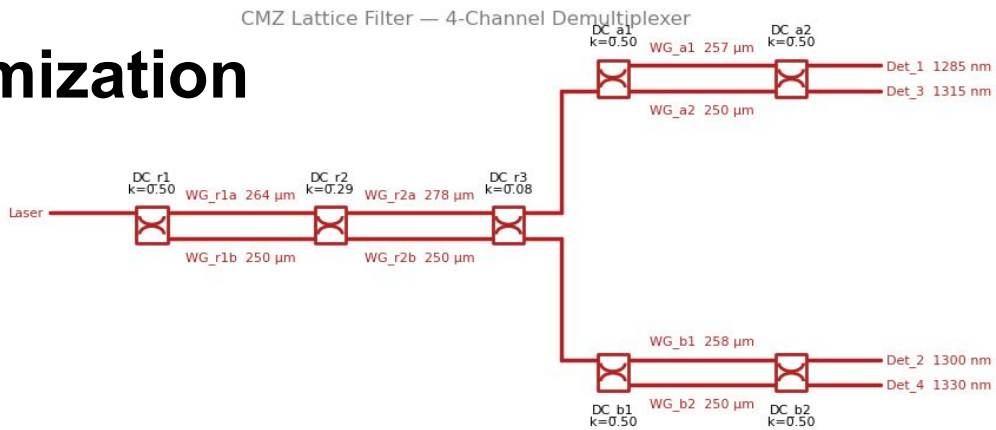


CPU-bound

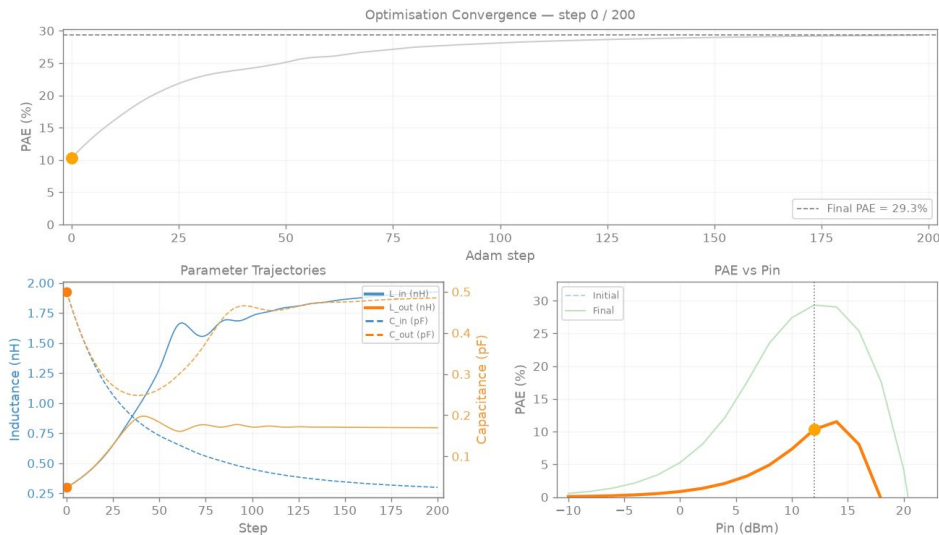


Agnostic
(CPU/GPU/TPU)

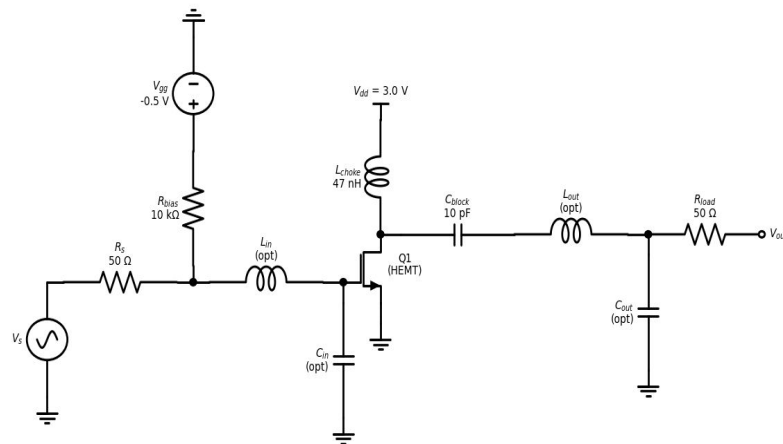
Photonic CMZ Demux Optimization



HEMT-PA Optimization via Differentiable Harmonic Balance



5 GHz Class-A HEMT PA — Circuit Topology



IHP PDK Architecture

gdsfactory.github.io/IHP

A modern, open Python-native photonics PDK

```
ihp-pdk/  
├── layers.py  
├── cross_sections.py  
├── technology.py  
├── cells/  
├── models/  
├── drc/  
├── examples/  
└── tests/
```



Everything is Python.

Readable. Hackable. Extensible.



Everything is version controlled.

Traceable history. Collaborative by design.



Everything is testable.

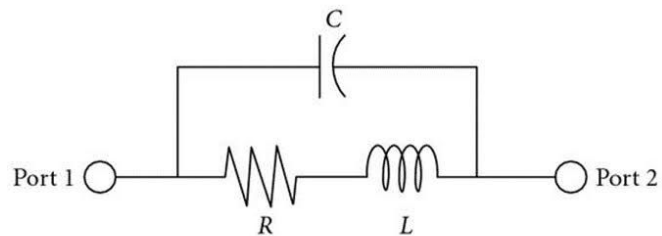
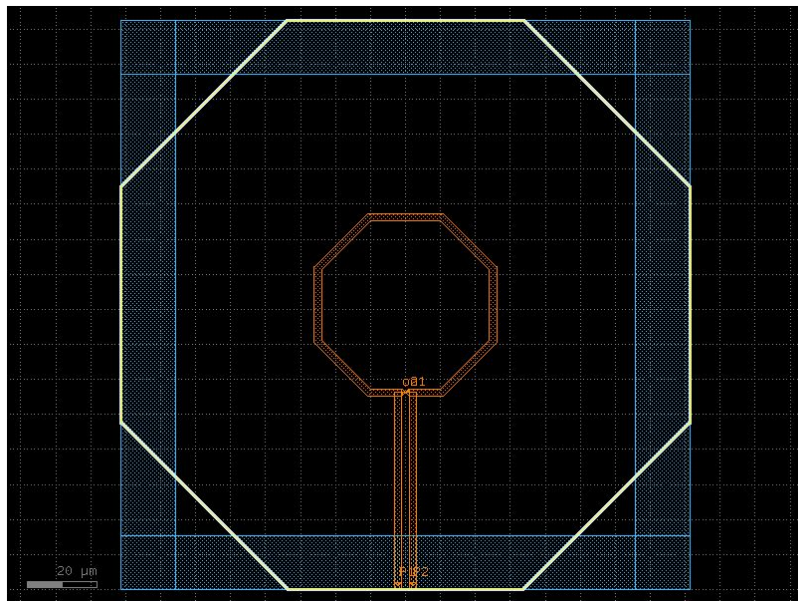
Automated tests ensure reliability across the stack.



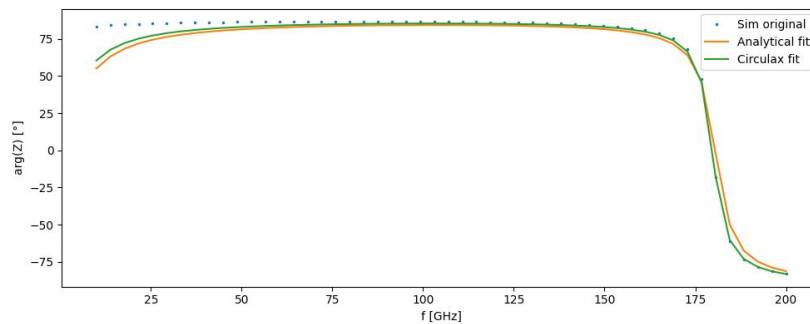
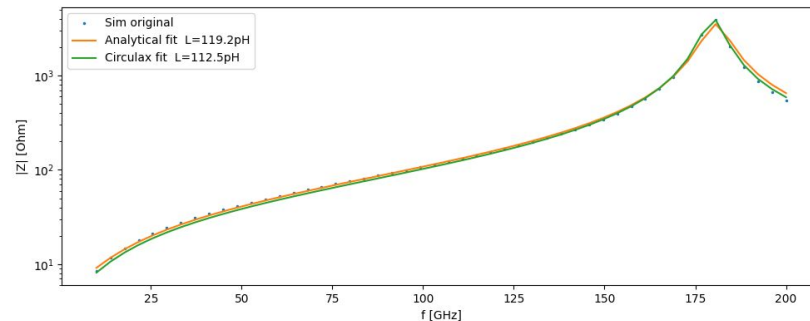
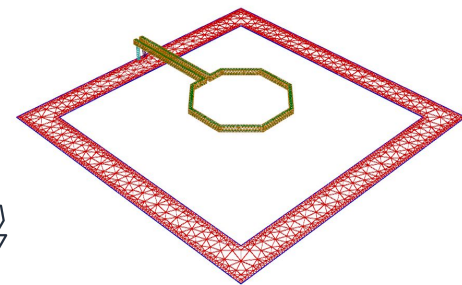
Open. Modular. Tested. Extensible.

Built for collaboration. Ready for innovation.

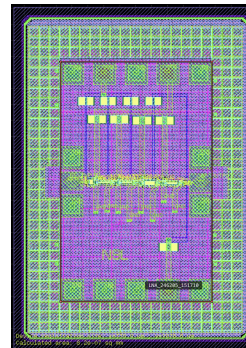
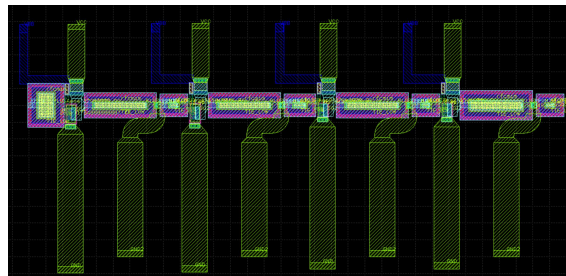
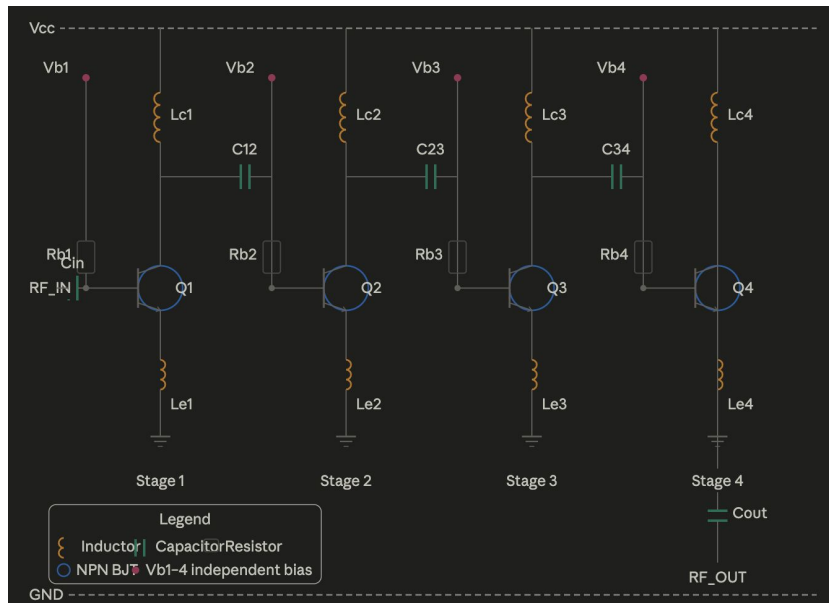
Inductor layout and extraction



Polace



160GHz LNA Example



Center Frequency	160 GHz
-3 dB Bandwidth	27 GHz
Frequency Range	146-173 GHz
Peak Gain (S21)	12.5 dB @ 157 GHz
Noise Figure	5.77 dB @ 160 GHz
Input P1dB	-11.4 dBm @ 160GHz

ROADMAP



More Open PDKs

Expand device coverage across more technologies.



More open packaging ADKs

Streamline packaging and assembly flows.



Better simulation models

Deliver more accurate, efficient compact models.



AI agents

Improve intelligent agents to accelerate design.



Analog generators

Broader analog IP for faster design.



More open-source collaborations

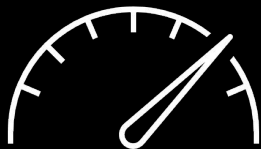
Grow the ecosystem through open collaboration.



Building the future of open EDA, together.

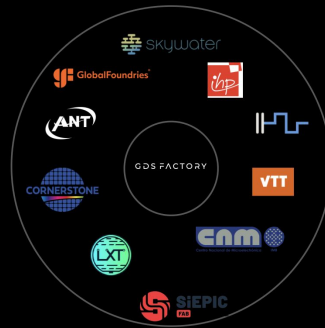
Summary

UP TO
200X FASTER
THAN GDSPy



FULLY PARAMETRIC IHP PCELL LIB
READY-TO-USE

10 OSS PDKs
AVAILABLE



SEAMLESS INTEGRATION WITH
20+ OPEN TOOLS

Questions?

gdsfactory.com

gdsfactory.com/careers

Questions?

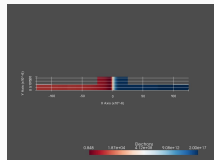
Backup

Device Solvers

From TCAD to FDTD – a complete toolbox for modern photonic and electronic design

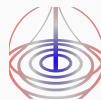
TCAD

Semiconductor devices

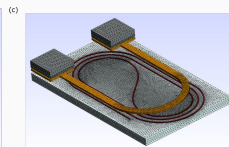
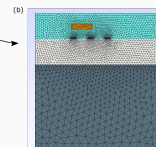
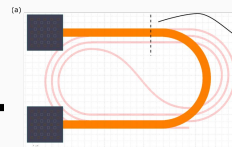


FEM

Finite-element multiphysics

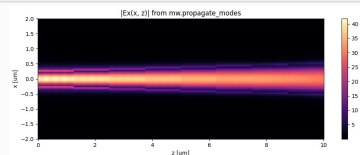
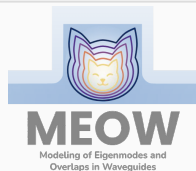


FEMWELL



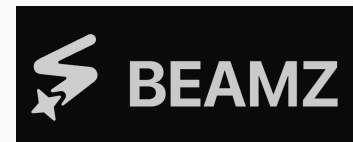
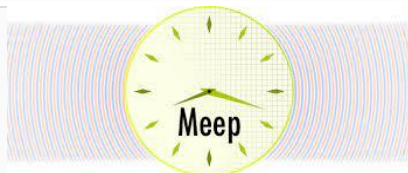
FDFD

Frequency-domain
Electromagnetics and
eigenmode expansion



FDTD

Time-domain
electromagnetics



Circuit solvers

Fast, accurate, and open-source tools for photonic and electronic circuit simulation

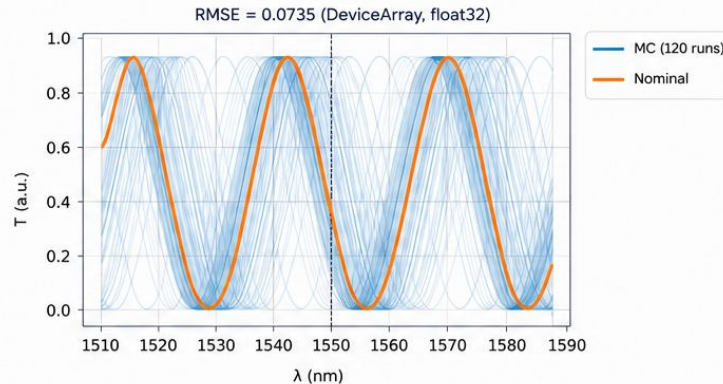
Frequency domain

S-parameter-based circuit simulation



sax

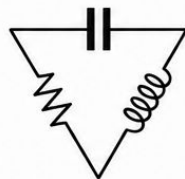
Frequency-domain solver
(S-parameter based)



- S-parameter analysis
- Multiport networks
- Statistical sweeps

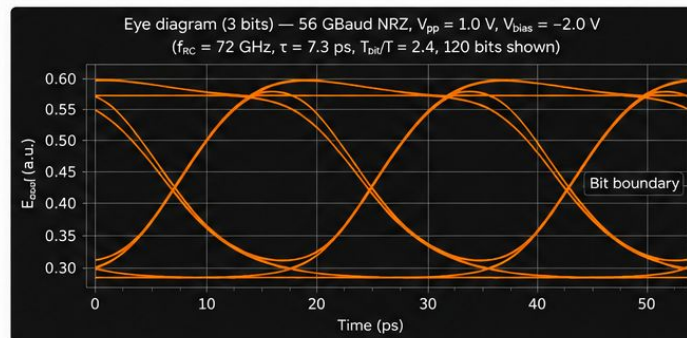
Time domain

Transient and time-resolved circuit simulation



circulax

Time-domain solver



- Transient analysis
- Eye diagrams
- High-speed signals

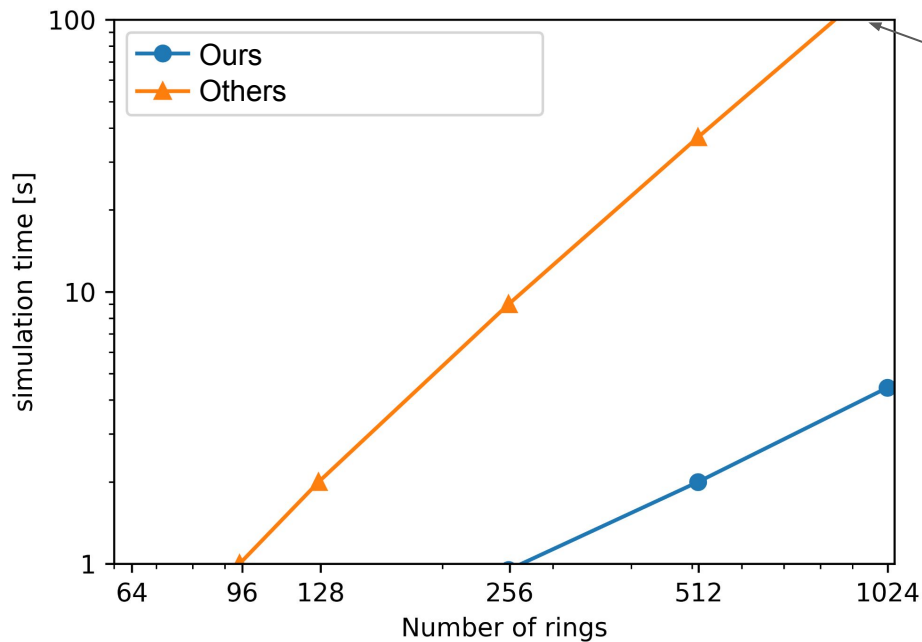
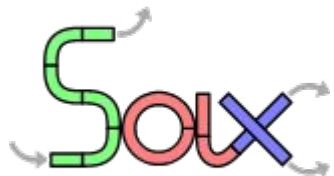
 High performance
Optimized solvers

 Open source
Transparent and extensible

 Seamless integration
Workflows in Python

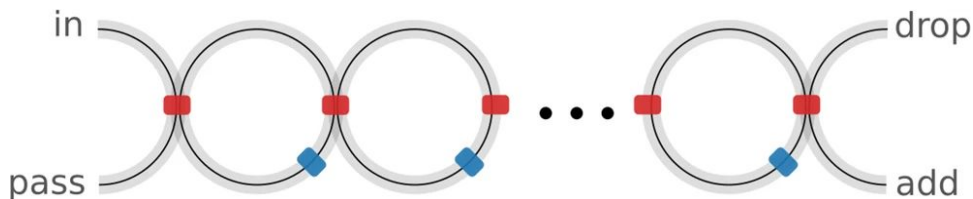
 Photonic & electronic
Unified simulation stack

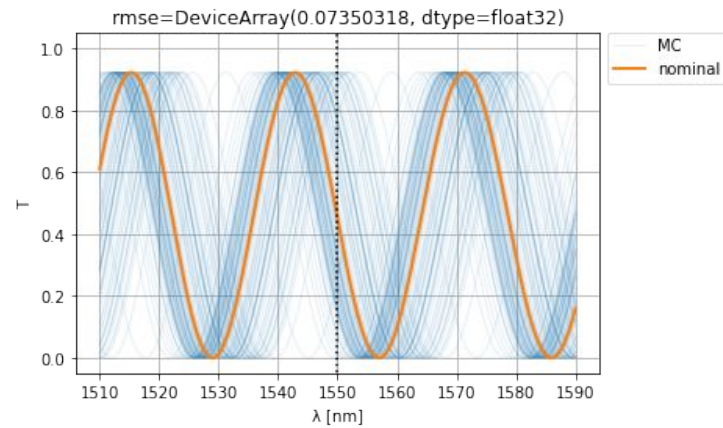
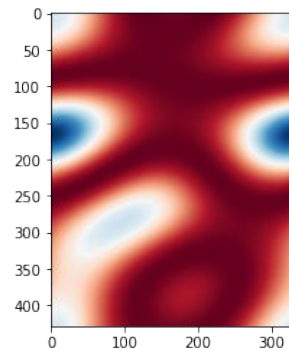
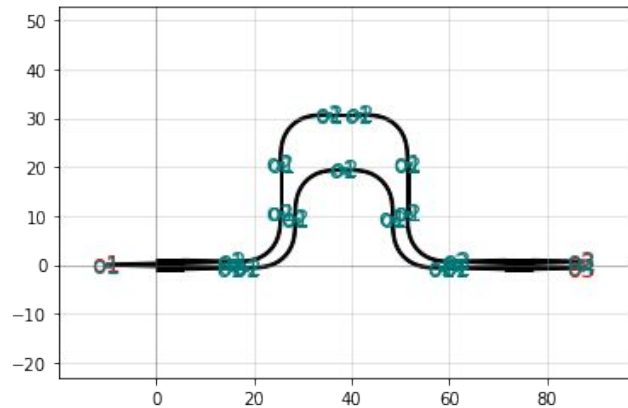
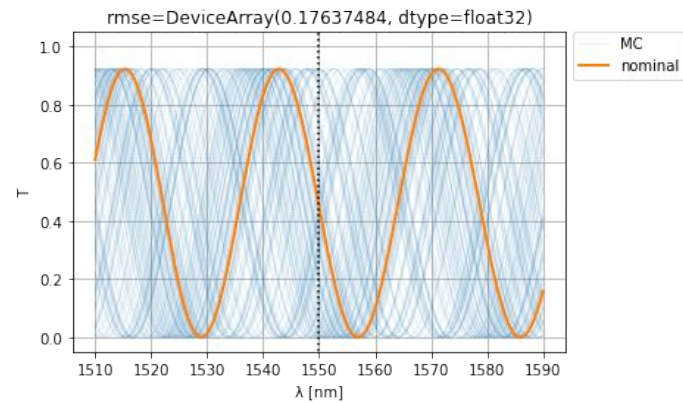
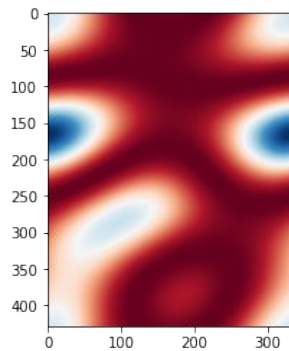
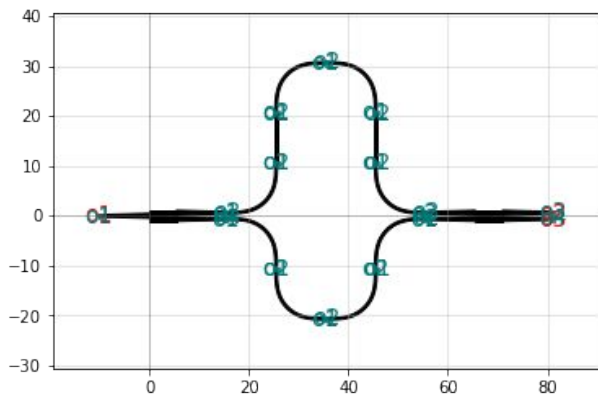
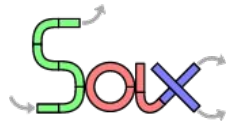
Benchmarks

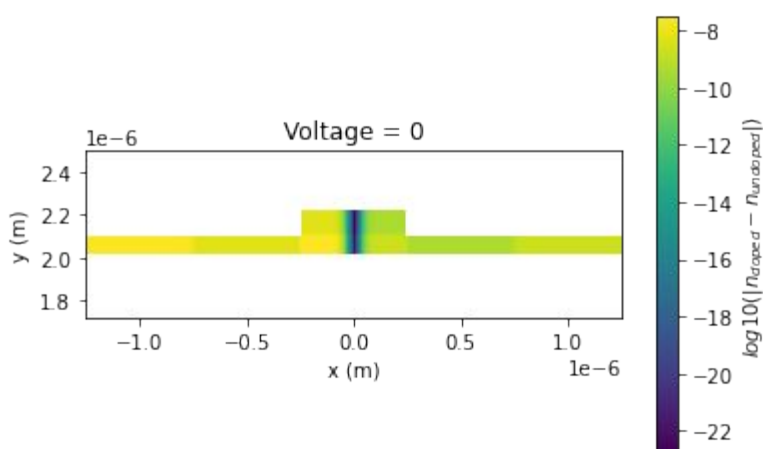
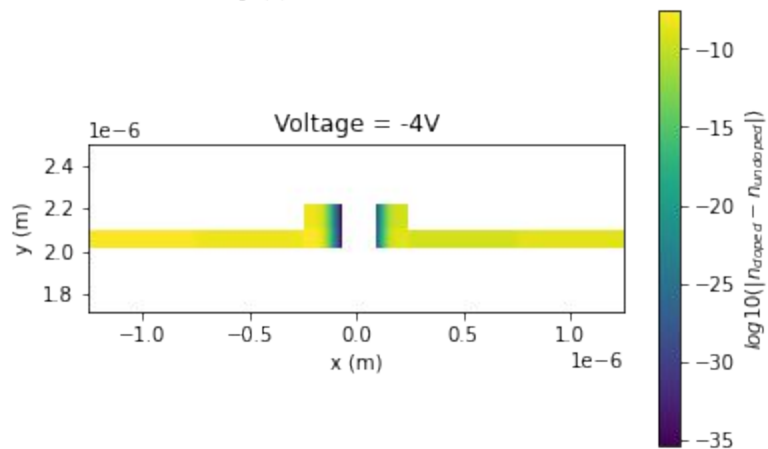
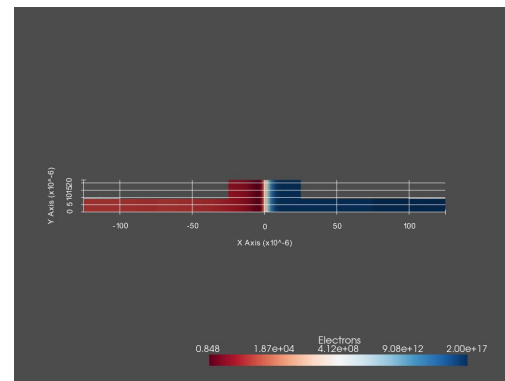
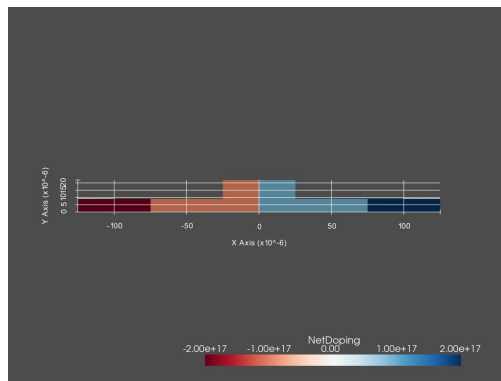
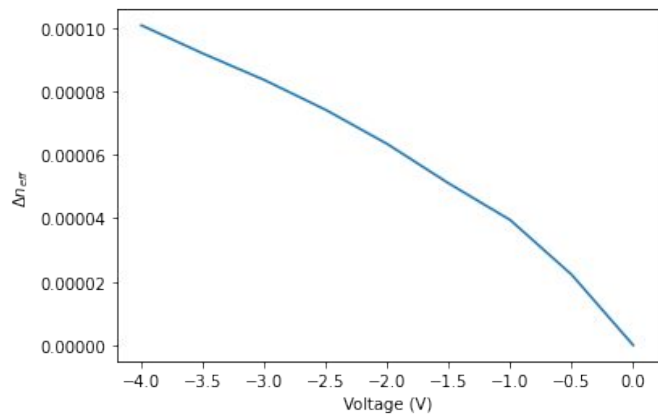


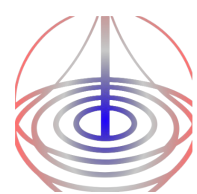
Others for circuits larger than 512 elements

SAX can handle large circuits thanks to a **sparse matrix solver**



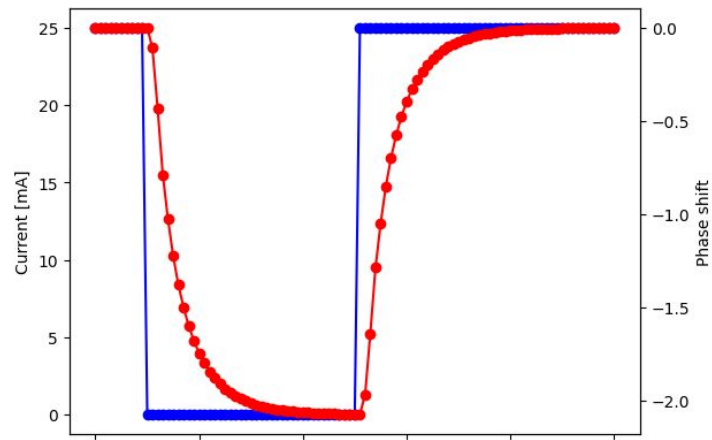
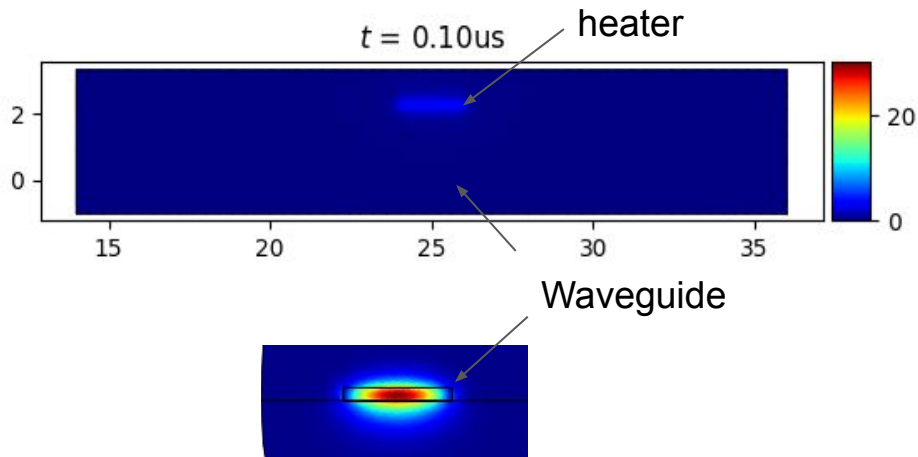
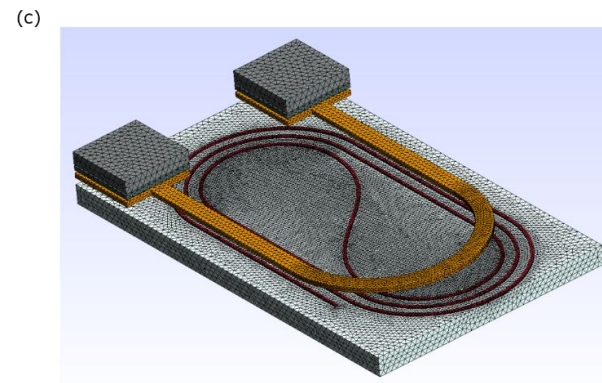
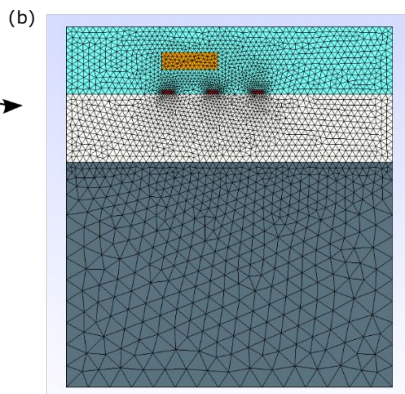
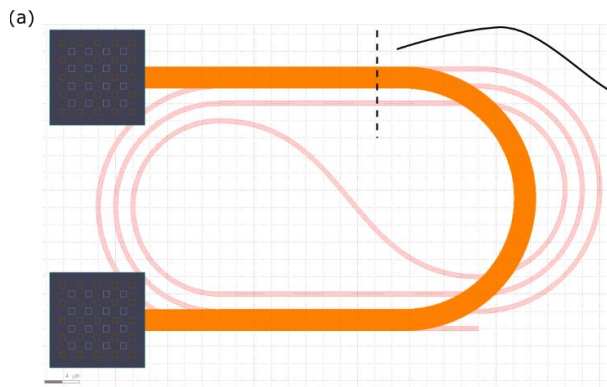


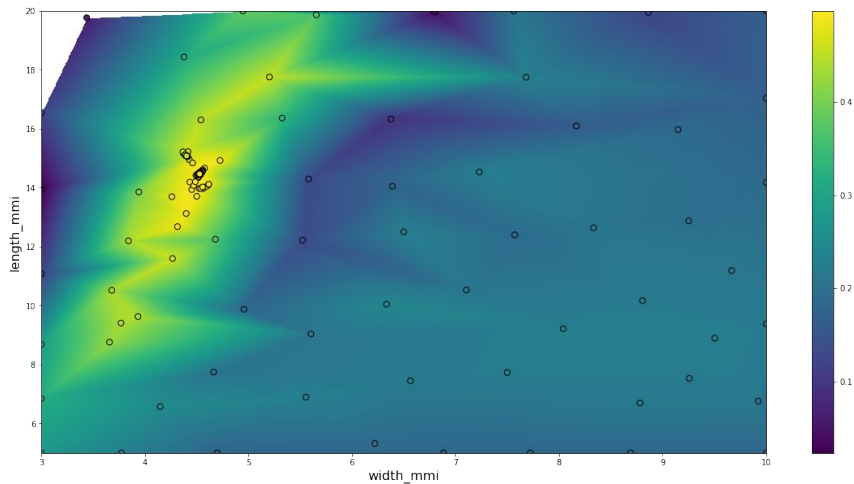
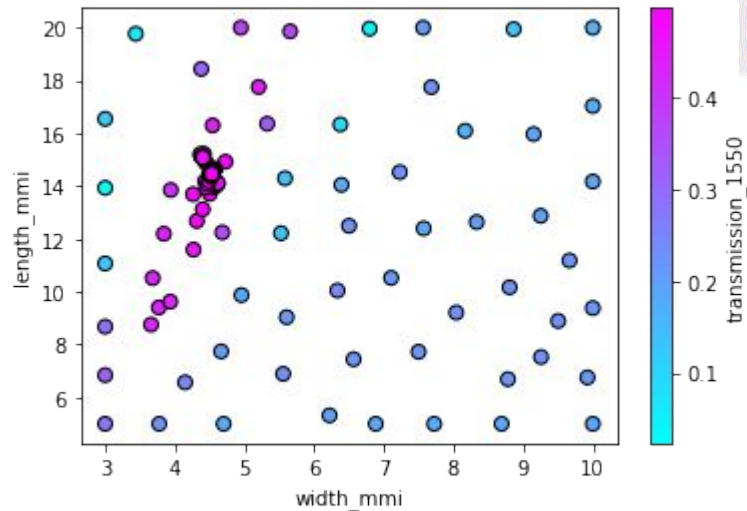
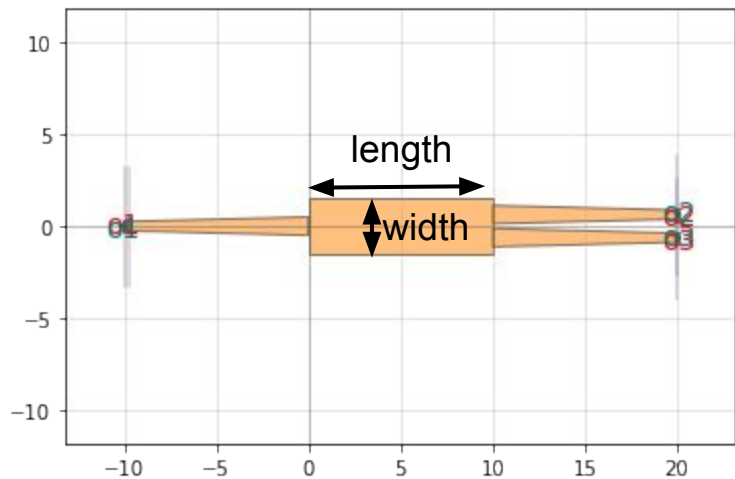




FEMWELL

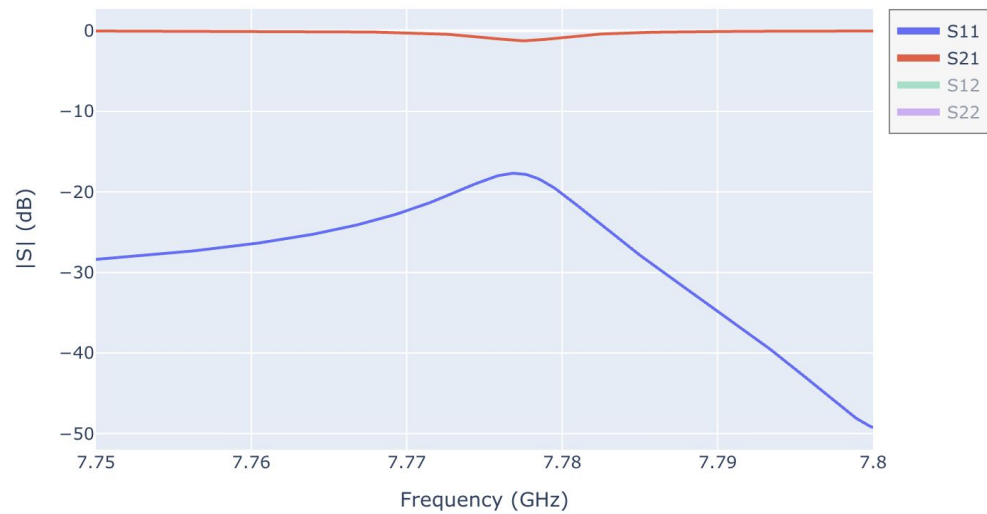
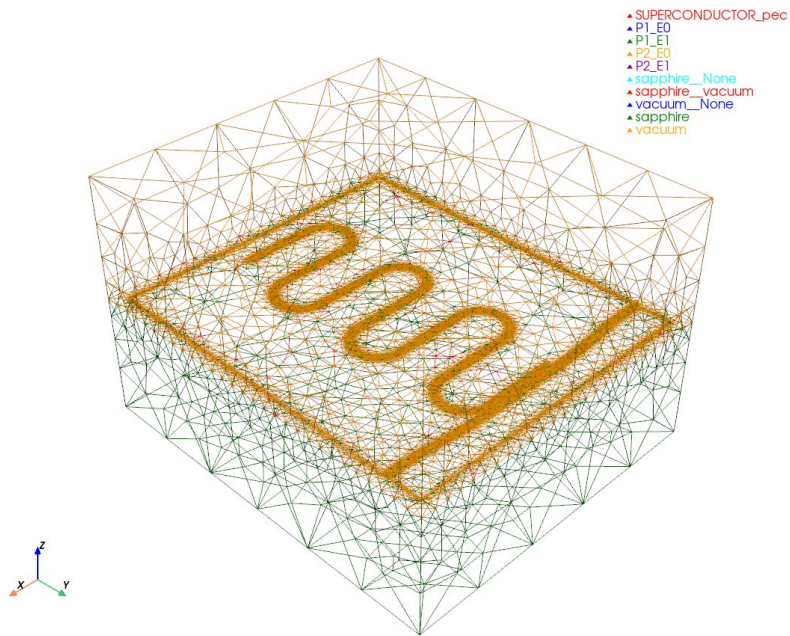
Multi-physics FEM solver



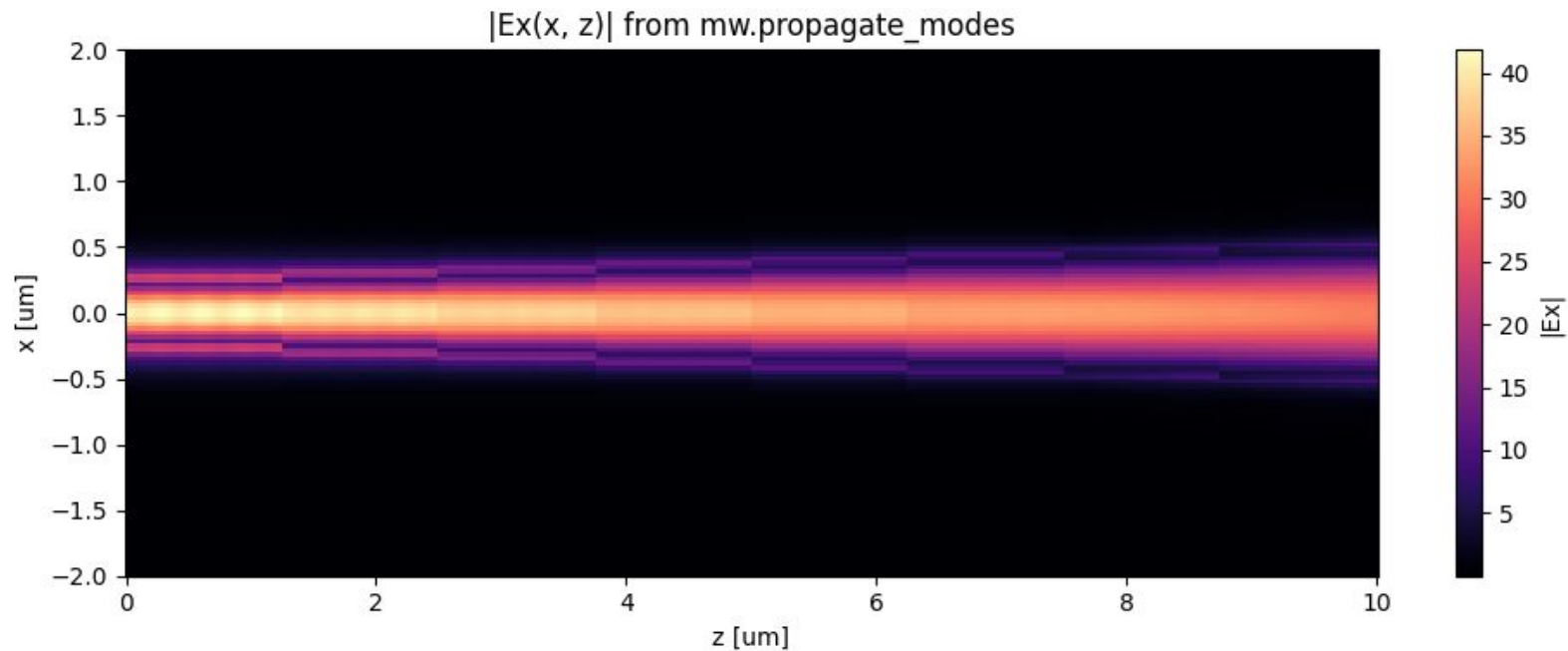
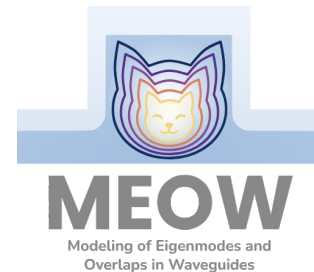
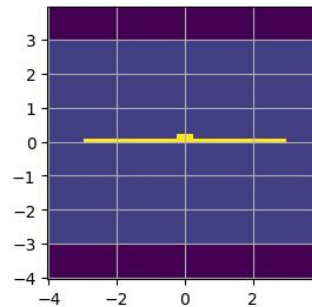
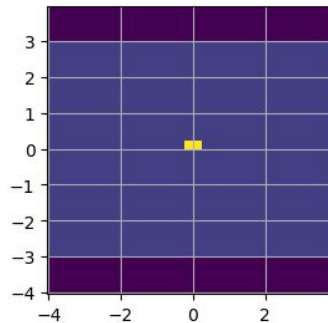


TRIAL_ID	width	length	transmission
128	3.000000	13.936181	0.023320
92	6.796302	19.966311	0.042681
150	4.535979	14.461443	0.497184
148	4.536362	14.458428	0.497189
86	4.531666	14.452401	0.497239
144	4.535423	14.446593	0.497312
104	4.555169	14.544551	0.497325
163	4.535932	14.450433	0.497512
167	4.535889	14.454474	0.497642

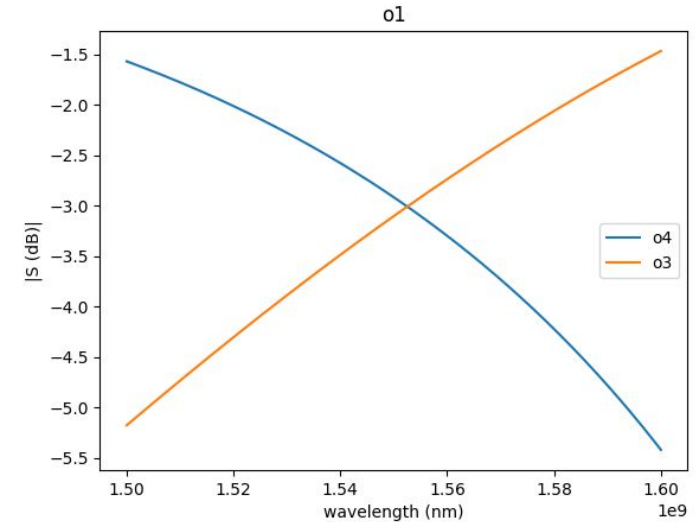
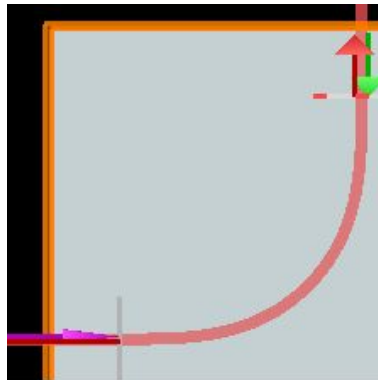
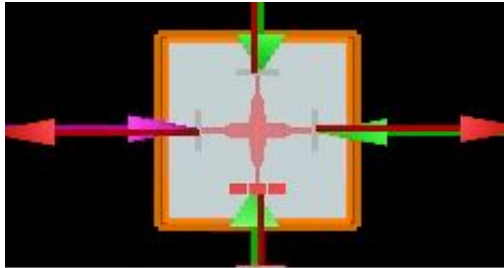
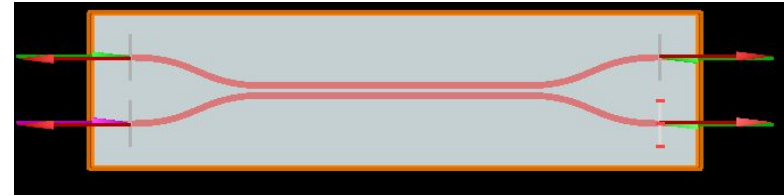
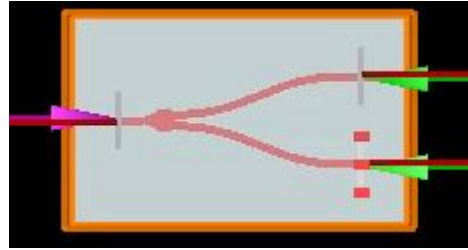
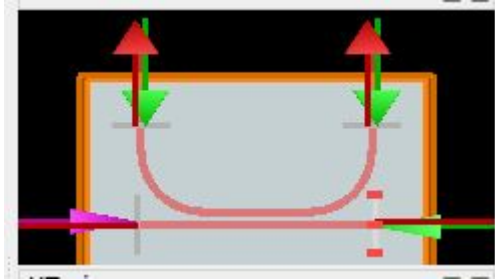
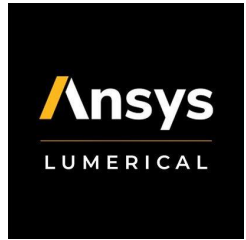
Polace



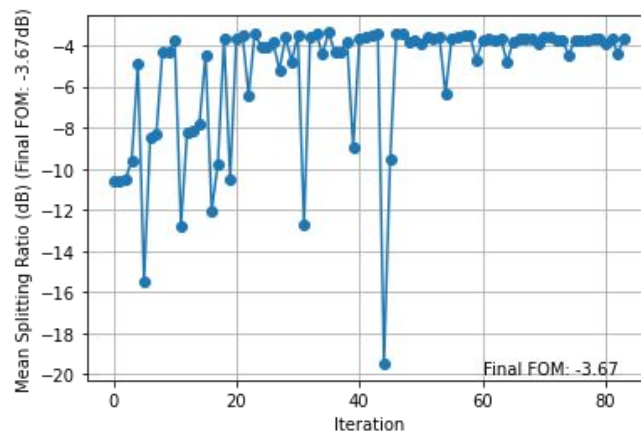
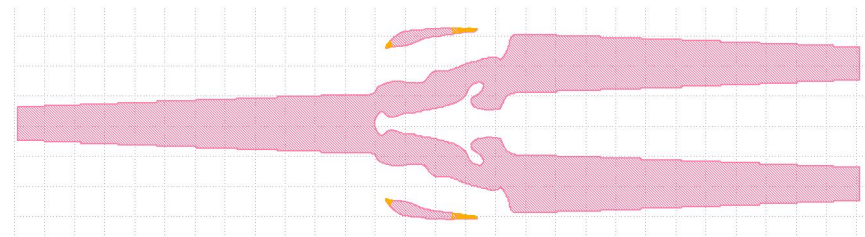
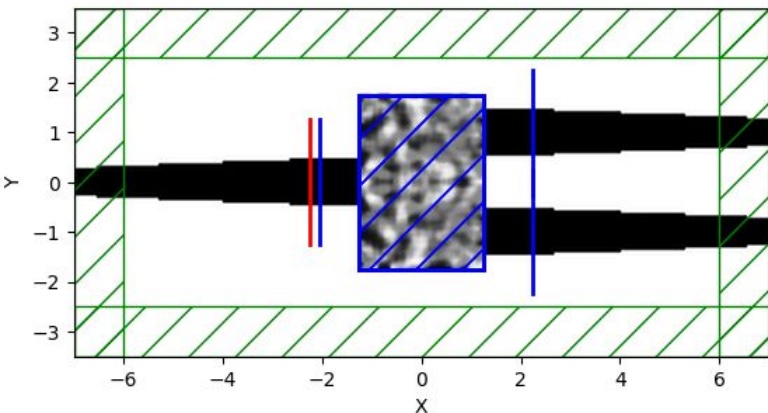
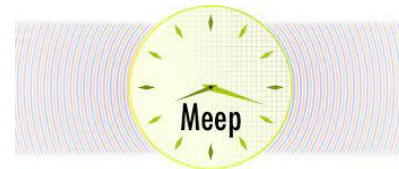
EigenMode Expansion



Automated FDTD Simulation from layout

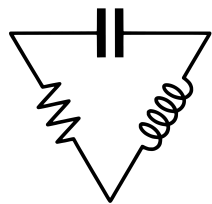
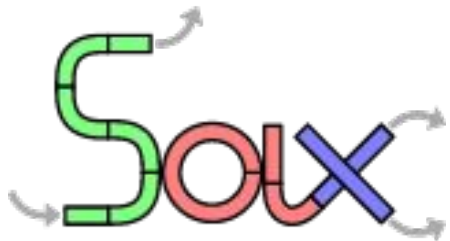


Adjoint optimization (inverse design)



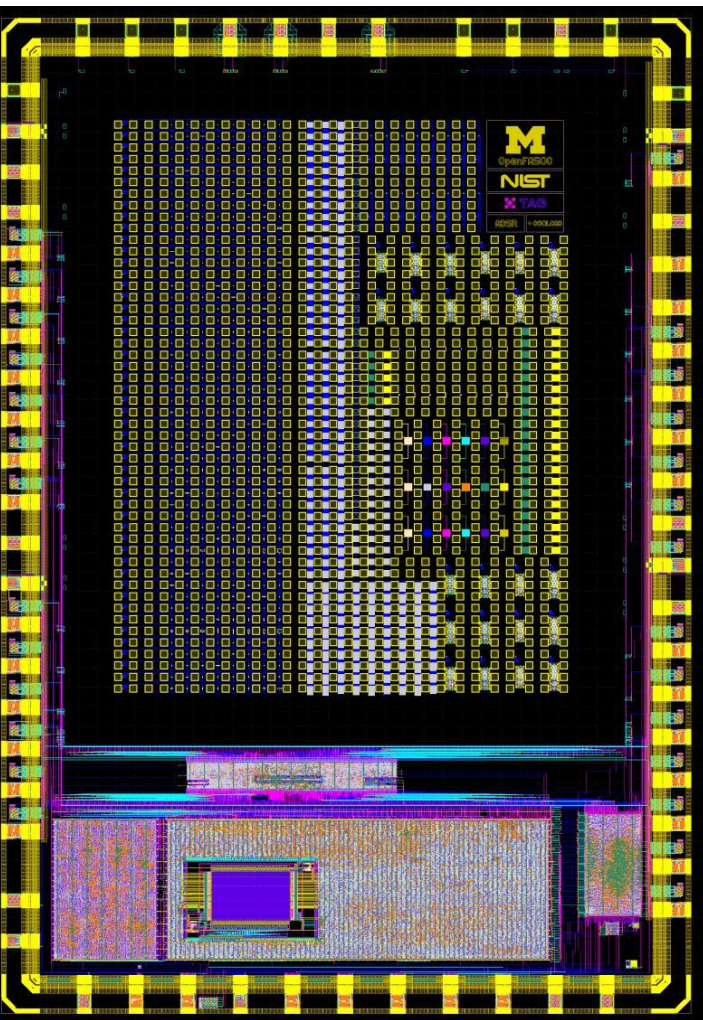
SAX and JAX: Fast, Pythonic, and Open Source!

- J: Just-In-Time Compilation
- A: Arithmetic That's Differentiable
- X: Cross-Platform Execution (CPU, GPU and TPU)



circularax





Tuohang Zeng • 1st

MS ECE student at the University of Michigan

6mo • Edited •



Tape-out submitted! I am excited to complete my first tape-out with [Mehdi Saligane](#) as my advisor at the [University of Michigan](#). This work was done in collaboration with [Brian Hoskins](#) at the [National Institute of Standards and Technology \(NIST\)](#), [David Fleischer](#) at [ADSR, Ltd.](#), and [Akin Akturk](#) at [CoolCAD Electronics LLC](#). Special thanks to [Tim Ansell](#) and [Google](#) for sponsoring the [SkyWater Technology Foundry](#), [Google](#), [Efabless Corporation](#) MPW program!

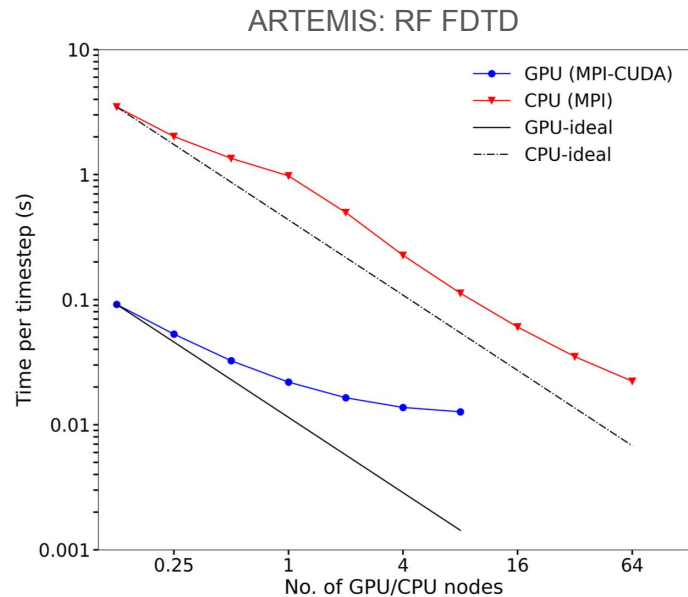
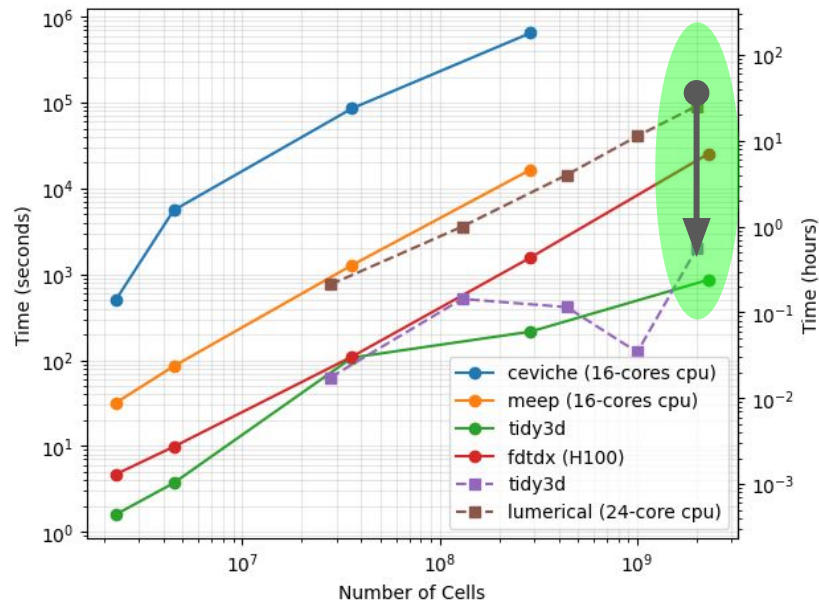
Our design consists of test structures on the open-source SKY130 PDK. Few stats to highlight:

- ~1400 bare pads
- 400+ transistor test structures
- 30 capacitor test structures
- 24 Ring Oscillators, built on 12 standard cell libraries using OpenFASOC
- 18 line resistance and via chain test structures
- ...

We completed our designs using open-source tools like Magic and KLayout, and heavily used automated flows like OpenFASOC, sitting on top of OpenROAD, and gdsfactory. The goal is to create open-source models at cryogenic temperature and enhance the existing SKY130 models, especially for high-end analog design.

Open-source hardware/EDA has enabled a new level of collaboration in IC design, and we are excited about what open-source PDKs and tools will empower us to do in the future!

FDTD benchmarks



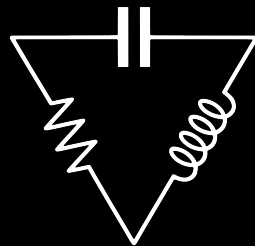
Circuit solvers

[SAX](#): S-Matrices with Autograd and XLA - a scatter parameter circuit simulator and optimizer for the frequency domain based on [JAX](#).



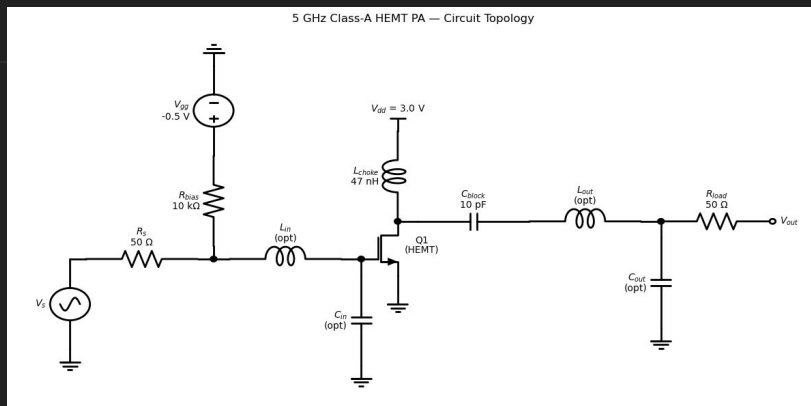
[Circulax](#) (new): Compliments [SAX](#) offering non-linear circuit simulations with multiple backends. State of the art optimization via back-propagation

- Time domain
- DC/AC analysis
- Harmonic balance



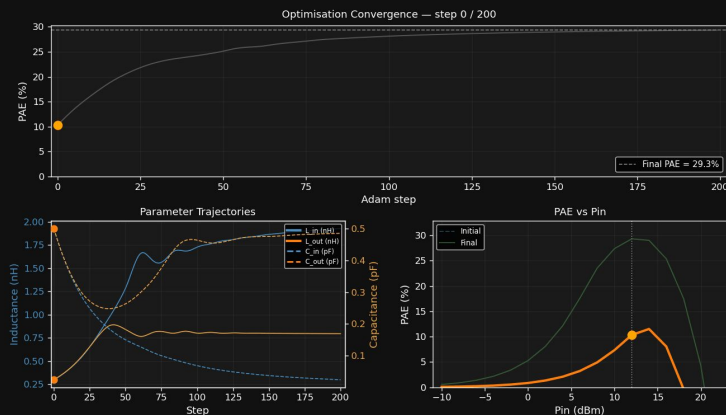
circulax

Circulax Optimization

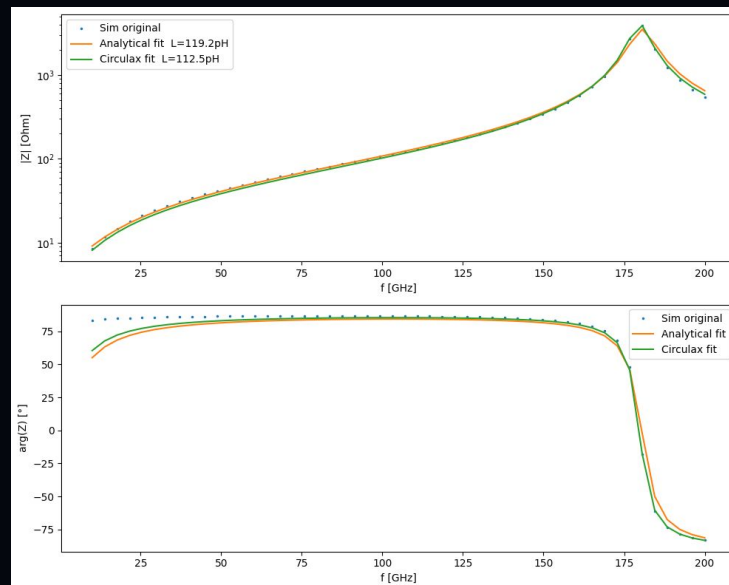
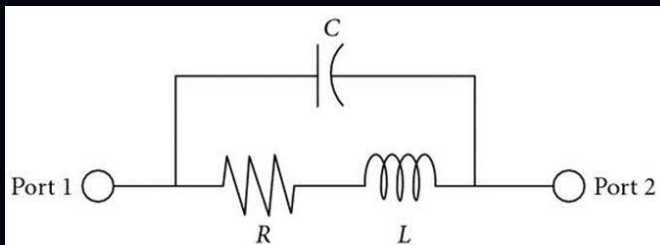
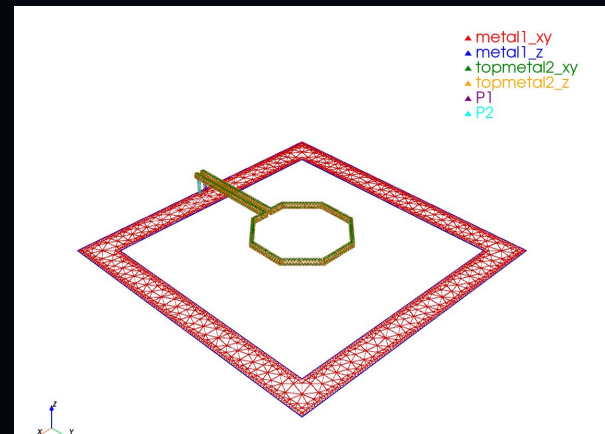
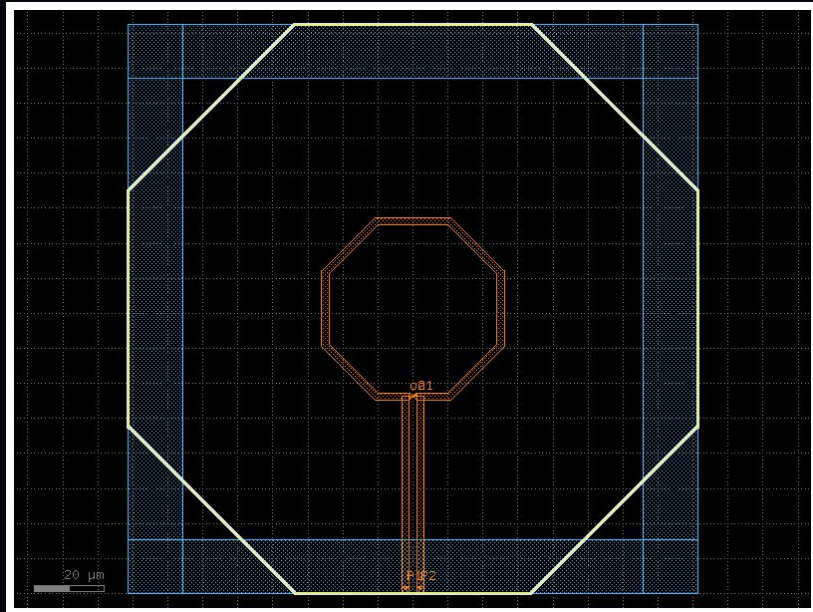


Back-prop (Adjoint) vs. Finite Difference in HB

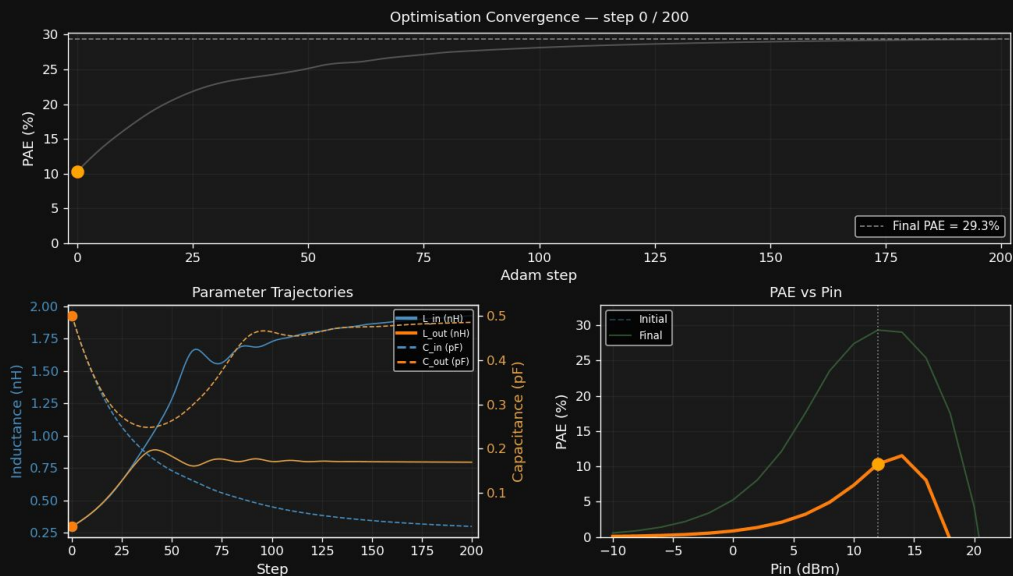
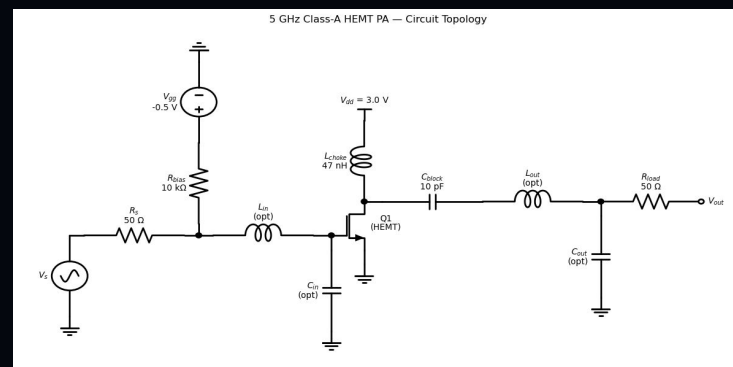
Feature	Back-propagation (Adjoint)	Finite Difference
Simulation Cost	Constant: 1 HB solve + 1 Linear solve	Linear: $N + 1$ full HB solves
Parameter Scaling	Optimized for 1,000s of variables	Becomes unusable after ~10 variables
Gradient Quality	Exact: Analytical precision	Noisy: Sensitive to step-size (h)
Optimization	Enables Gradient Descent / Adam	Limited to derivative-free or slow solvers



Inductor layout and extraction



Power amplifier



Tech

1. Python Code



```
@gf.cell
def hello_world(size_text: float = 10) -> gf.Component:
    """Simple Hello World PCell.

    Args:
        size_text: Text size in microns.
    """
    c = gf.Component()

    ref1 = c.add_ref(
        gf.components.rectangle(size=(10, 10), layer=(1, 0))
    )
    ref2 = c.add_ref(
        gf.components.text("Hello", size=size_text, layer=(2, 0))
    )
    ref3 = c.add_ref(
        gf.components.text("world", size=size_text, layer=(2, 0))
    )

    ref1.xmax = ref2.xmin - 5
    ref3.xmin = ref2.xmax + 2
    ref3.rotate(30)

    return c
```



2. Write CAD (GDS, OASIS, STL, GERBER)



3. Run simulations

